EE/CS 148B HW 4

Diffusion Models

Lead TAs: Ziqi Ma & Aadarsh Sahoo

Spring 2026

1 Assignment Overview

In this assignment, you will build and analyze diffusion models from the ground up. You will first derive the mathematical foundations of DDPM and score matching, then implement a VP-SDE score model and train it on FashionMNIST, and finally compare it against Rectified Flow—a conceptually simpler generative framework based on straight-line trajectories. Along the way, you will connect the iterative denoising, score matching, and SDE perspectives, and scale up to 256×256 image generation using OpenAI's guided diffusion codebase.

What you will implement.

1. The DDPM training objective and its derivation (§2)
2. Score matching techniques and Langevin dynamics (§3)
3. The SDE perspective of diffusion models and its connection to DDPM (§4)
4. Rectified Flow—theory and comparison to DDPM (§5)
5. A VP-SDE score model: forward process, EM sampler, and PC sampler (§6)
6. A Rectified Flow model: Euler sampler, reflow, and quantitative comparison (§7)
7. Guided diffusion on 256×256 images (§8)

What the code looks like. All assignment code is hosted on GitHub at `https://github.com/caltech-eecs148b/hw4`. For details on the repository layout, please refer to the `README.md`. The starter code provides a time-conditioned U-Net (shared by Parts 5 and 6), training loop scaffolding, and test hooks. Each implementation section starts with a **Files to update** box; use those as the authoritative map from problems to starter-code files.

Quick file map.

- §2–§5: written problems only (no code)
- §6: `diffusion/vp.py`, `scripts/train_vp.py`
- §7: `diffusion/rectflow.py`, `scripts/train_rectflow.py`, `scripts/sample.py`, `scripts/eval_kid.py`
- §8: `scripts/guided_diffusion_experiments.py`

Where to get the data.

- **FashionMNIST** (28×28 grayscale images, 10 clothing classes). Downloaded automatically by `torchvision` the first time you run `scripts/train_vp.py` or `scripts/train_rectflow.py`. No manual setup required.
- **Guided diffusion model weights** (Part 7 only). Clone the OpenAI guided-diffusion repository at https://github.com/openai/guided-diffusion and download `256x256_diffusion_uncond.pt` and `256x256_classifier.pt` from the model card. Place them under `guided-diffusion/models/`. See `data/README.md` for details.

How to submit. You will submit the following files to Gradescope:

- `assignment4.pdf`: Answer all written questions. Please typeset your responses. Include a link to your Colab notebook at the top of the file.
- `VP.py`: Your implementation of the VP SDE, submitted to **HW4 — Code** on Gradescope.

Note: we must be able to run both your Colab notebook and your submitted code.

Note on Compute. We assume students have access to Colab Pro+. Please keep your receipts for end-of-quarter reimbursement. Alternatively, use any other compute you have access to.

- For the written problems (§2–§5) and the coefficient plot (Problem 1.8), no GPU is needed.
- For VP and Rectified Flow training (§6–§7), use an **A100**. Each training run takes approximately 10 minutes.
- For guided diffusion (§8), use an **A100**. Each sampling run takes 5–10 minutes.

2 Part 1: DDPM — The Iterative Denoising Perspective

In this part, we will delve deep into the Denoising Diffusion Probabilistic Models (DDPM) paper [1] and understand the mathematical formulation behind it.

Problem 1.1: Reading

Read through the paper [1]. Make sure you understand the main idea of the paper. The rest of this part will guide you through the derivations of DDPM. In later parts, we follow the notations in the paper.
No written answer required.

Problem 1.2: Variational Bound

In the Background section of the paper, the authors wrote "the training [of diffusion models] is performed by optimizing the usual variational bound on negative log likelihood." Let's go into more details. Explain why we want to minimize the variational bound on negative log likelihood and prove the chain of (in)equalities in Equation (3).
Deliverable: Your proof and explanation.

Problem 1.3: Tractable Loss Function

The construction of a tractable loss function is one of the most important parts of the original diffusion model paper [2] and DDPM. The derivation is given in Appendix A of [1], but please provide more mathematical details to justify why Equations (18), (20), (21), and (22) hold. These quantities involve simplifying terms, invoking established equalities/theorems, and using definitions. Please write out what equalities/theorems/definitions are being used for each of those steps.
Deliverable: Detailed justification for each equation.

Problem 1.4: Markov Process — Equation (4)

The derivation of B and C is general, but now let's focus on the Markov process of the DDPM. First prove Equation (4).
Deliverable: Your proof.

Problem 1.5: Markov Process — Equations (6) and (7)

Then prove Equations (6) and (7).
Hint: Bayes' rule is one way to do it, but there is a simpler way. Consider the joint distribution of x_{t-1} and x_t conditioned on x_0, and then use the Gaussian conditioning formula. Try using the reparametrization trick to write x_t in terms of x_{t-1}. This list of covariance properties can be useful.
Deliverable: Your proof.

Problem 1.6: KL Divergence and Equation (8)

The KL divergence between two k-dimensional multivariate Gaussian distributions $p(\boldsymbol{\mu}_p, \boldsymbol{\Sigma}_p)$ and $q(\boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q)$ can be computed in closed form as follows:

$$D_{\mathrm{KL}}(p \,\|\, q) = \frac{1}{2}\left[\log\frac{|\boldsymbol{\Sigma}_q|}{|\boldsymbol{\Sigma}_p|} - k + \mathrm{tr}(\boldsymbol{\Sigma}_q^{-1}\boldsymbol{\Sigma}_p) + (\boldsymbol{\mu}_q - \boldsymbol{\mu}_p)^\top \boldsymbol{\Sigma}_q^{-1}(\boldsymbol{\mu}_q - \boldsymbol{\mu}_p)\right].$$

Assume that $\boldsymbol{\Sigma}_\theta(x_t, t) = \beta_t \mathbf{I}$ where β_t is defined in Equation (7). Use the above fact to prove Equation (8):

$$L_{t-1} - C = \mathbb{E}_q\left[\frac{1}{2\beta_t}\left\|\tilde{\boldsymbol{\mu}}_t(x_t, x_0) - \boldsymbol{\mu}_\theta(x_t, t)\right\|^2\right],$$

where C is a constant that does not depend on θ.
Deliverable: Your proof.

Problem 1.7: Parameterization and Equation (12)

Plug the parameterization in Equation (11) into Equation (10) and show that $L_{t-1} - C$ is indeed equal to Equation (12):

$$\mathbb{E}_{x_0,\boldsymbol{\epsilon}}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\left(\sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},\, t\right)\right\|^2\right].$$

Deliverable: Your derivation.

Problem 1.8: Coefficient Plot

Assume β_t defined in Section 4 and $\sigma_t^2 = \beta_t$ defined in Equation (7). Plot the coefficient (**y-axis in log scale**)

$$\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}$$

with respect to t (x-axis). Note how the coefficient is relatively smaller for large t. This observation means that the simplified objective, which drops the coefficients, is equivalent to down-weighting the loss terms for smaller t. Empirically, this leads to better performance. See a discussion in Section 3.4 of the paper.

Deliverable: Your plot.

3 Part 2: Score Matching and Langevin Dynamics

In this part, we are going to shift our perspective from denoising to score functions. By definition, the (Stein) score function of a probability distribution with density $p(x)$ is $\nabla \log p(x)$, where the gradient is taken with respect to x.

Problem 2.1: Reading

Read through the paper [3]. Make sure you understand the main idea. In the following parts, we will learn more about some of the score matching techniques mentioned in the paper.
No written answer required.

Problem 2.2: Explicit Score Matching

In score matching, one aims to use a neural network to approximate the score function. Naively, one can write down the following optimization objective, which is called the explicit score matching (ESM) formulation:

$$\mathcal{J}_{\mathrm{ESM}}(\theta) = \mathbb{E}_{p(x)}\left[\frac{1}{2}\left\|s_\theta(x) - \nabla_x \log p(x)\right\|^2\right]. \tag{1}$$

Explain why this particular formulation is often impossible to work with in many application scenarios of diffusion models.
Deliverable: A brief explanation.

Problem 2.3: Implicit Score Matching

Prove that objective (1) is equivalent to the following objective, i.e. they differ only by a constant term that does not depend on the parameter θ:

$$\mathcal{J}_{\mathrm{ISM}}(\theta) = \mathbb{E}_{p(x)}\left[\mathrm{tr}(\nabla_x s_\theta(x)) + \frac{1}{2}\left\|s_\theta(x)\right\|^2\right], \tag{2}$$

where $\nabla_x s_\theta(x)$ is the Jacobian of $s_\theta(x)$ with respect to its input x. This formulation is called the implicit score matching (ISM).
Hint: you need to use vector integration by parts and the fact that $p(x)$ vanishes at infinity.
Deliverable: Your proof.

Problem 2.4: Limitation of ISM

Explain why objective (2), despite being tractable to compute, may not be a desirable optimization objective to work with.
Deliverable: A brief explanation.

Problem 2.5: Tweedie's Formula

Now we focus on denoising score matching (DSM). Prove Tweedie's formula:

$$\mathbb{E}[x \mid \tilde{x}] = \tilde{x} + \sigma^2 \nabla \log p_\sigma(\tilde{x}),$$

where $\tilde{x} = x + \sigma\epsilon$, $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, $x \sim p(x)$.
Hint: Use the following steps:

- Move $\tilde{x}$ to the left-hand side and use $\tilde{x} = \mathbb{E}[\tilde{x} \mid \tilde{x}]$ to simplify.
- Write the conditional expectation as an integral.
- Apply Bayes' rule to show that the left-hand side can be written as $\int (x - \tilde{x})\, p(x \mid \tilde{x})\, dx$.
- Show that $\nabla \log p_\sigma(\tilde{x}) = \int \nabla_{\tilde{x}} p(x \mid \tilde{x}) \frac{p(x)}{p(\tilde{x})}\, dx$.
- Expand and rearrange terms. Think about what $\nabla_{\tilde{x}} p(x \mid \tilde{x})$ is in this case.

Deliverable: Your proof.

Problem 2.6: DSM Objective from Tweedie's Formula

Use Tweedie's formula to establish a training objective for score matching. Then prove that the established training objective is equivalent to the DSM objective:

$$\mathcal{J}_{\text{DSM}}(\theta; \sigma) = \mathbb{E}_{p(x)} \mathbb{E}_{p(\tilde{x}|x)} \left[\frac{1}{2} \left\| s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log p(\tilde{x} \mid x) \right\|^2 \right],$$

i.e. they differ only by a constant term that does not depend on θ.
Deliverable: Your derivation and proof.

Problem 2.7: DSM vs. DDPM

Compare the DSM objective with the simplified training objective at time t in DDPM with $x = \alpha_t x_0$, $\tilde{x} = x_t$, and $\sigma = \sqrt{1 - \bar{\alpha}_t}$. Show that the learned model $s_\theta(\tilde{x})$ in the DSM differs from the learned model $\epsilon_\theta(x_t, t)$ by a constant factor involving σ.
More precisely, show that there exists a scalar $c(\sigma) \in \mathbb{R}$, which depends on σ, such that if ϵ^*_θ is the optimizer for the DDPM objective at a certain time t, then $s_{\theta^*} := c(\sigma)\, \epsilon^*_\theta$ will be the optimizer for the DSM objective with $x = \alpha_t x_0$, $\tilde{x} = x_t$, and $\sigma = \sqrt{1 - \bar{\alpha}_t}$.
Deliverable: Your derivation.

Problem 2.8: Langevin Dynamics Sampling

Once you do the DSM for some noise level σ, you have an estimate of $p_\sigma(x)$. Consider the overdamped Langevin dynamic:

$$dx_t = \nabla \log p_\sigma(x_t)\, dt + \sqrt{2}\, dB_t,$$

where $\nabla \log p_\sigma(x)$ is the score function of $p_\sigma(x)$ and B_t is the n-dimensional Brownian motion. This SDE admits $p_\sigma(x)$ as its stationary distribution. Under weak regularity conditions, the distribution of x_T approaches $p_\sigma(x)$ as $T \to \infty$. When σ is small, $p_\sigma(x) \approx p(x)$. Based on these facts, describe an algorithm that approximately samples from the target distribution $p(x)$.
Deliverable: Your algorithm description.

4 Part 3: The SDE Perspective of Diffusion Models

In Part 2, we showed that DSM is almost equivalent to the training of DDPM. In this part, you will see that when the time discretization converges to 0, the forward process of DDPM is a diffusion SDE. Furthermore, the backward (sampling) process of DDPM is equivalent to solving the reverse SDE of the forward diffusion SDE involving the score function. So DDPM can be perfectly viewed from an SDE perspective!

Problem 3.1: Reading

Read through the paper [4]. Make sure you understand the main idea. In the following parts, you will prove some of the results in the paper.
No written answer required.

Problem 3.2: VP-SDE Approximations

In Appendix B, the authors show that in the limit of time discretization $\Delta t \to 0$, the forward process of DDPM is equivalent to the variance-preserving (VP) SDE:

$$dx = -\tfrac{1}{2}\beta(t)\, x\, dt + \sqrt{\beta(t)}\, dB_t$$

with $\beta(t)$ defined in [4]. The authors showed this via two approximations ($\approx$) and claimed that they hold when the step size $\Delta t \ll 1$. Explain why those two approximations hold.
Deliverable: Your explanation.

Problem 3.3: Reverse VP-SDE

A remarkable fact shown in [5] is that the reverse process of a diffusion SDE is also a diffusion SDE. The formula for the reverse SDE of a general SDE is shown in Equation (6) of [4]. Write down the reverse SDE of the VP-SDE accordingly.
Deliverable: The reverse SDE expression.

Problem 3.4: Reverse VP-SDE Discretization

Fix the discretization Δt and let β_i be defined in the same way as in Appendix B&D of [4] ($\beta_i = \bar{\beta}_i \Delta t$, $x_i = x_t$, $x_{i+1} = x_{t+\Delta t}$). Follow these steps to establish the equivalence between the reverse VP-SDE and DDPM sampling:

1. Discretize the SDE you wrote down in Problem 3.3.

2. Substitute the true score function by the approximation s_θ.

3. Arrange terms such that x_i is on one side and terms that involve s_θ, β_{i+1}, x_{i+1}, and z_{i+1} are on the other side.

4. Follow the derivation in Appendix E of [4] to conclude that your discretization of the VP-SDE equals the following update rule when $\beta_i \to 0$:

$$x_i = \frac{1}{\sqrt{1-\beta_{i+1}}}(x_{i+1} + \beta_{i+1}\, s_\theta(x_{i+1}, i+1)) + \sqrt{\beta_{i+1}}\, z_{i+1}.$$

Deliverable: Your derivation.

Problem 3.5: Connecting SDE and DDPM

This update rule is almost the same as that in Algorithm 2 of [1] when the σ_t variable is chosen to be $\sigma_t = \sqrt{\beta_t}$. The only difference is the coefficient in front of s_θ. Use your answer in Problem 2.7 to justify this difference.
Hint: Think about what σ is in the context of DDPM. Note that σ is the standard deviation of the noise added to the clean image x_0.
Deliverable: Your justification.

Problem 3.6: Reverse SDE vs. Langevin Dynamics

Langevin dynamics was already used before diffusion models as a way to sample from a distribution by learning only the score function $\nabla \log p(x)$. Briefly explain why the reverse SDE approach leverages more information than Langevin dynamics for sampling.
Then explain how this extra information is reflected in both the score matching objective (Equation (7) of [4]) and the training objective of DDPM (Equation (14) of [1]).
Deliverable: A 2–3 sentence explanation for each point.

Problem 3.7: Conditional Diffusion and Classifier Guidance

Consider the following conditional reverse diffusion process on some conditional signal y:

$$dx = \left[f(x,t) - g(t)^2 \nabla_x \log p_t(x \mid y) \right] dt + g(t)\, d\bar{B}_t,$$

where $d\bar{B}_t$ is the time-reversed Brownian motion. The solution to this reversed SDE is $p(x \mid y)$. Using Bayes' rule, briefly explain the intuition of how classifier guidance diffusion works.
Note: if one uses a function $s_\theta(x_t, y, t)$ to approximate $\nabla_{x_t} \log p_t(x_t \mid y)$ via score matching and then samples the reverse SDE with $s_\theta(x_t, y, t)$, it is called classifier-free sampling.
Deliverable: Your explanation.

Problem 3.8: Probability Flow ODE (Optional Reading)

A lot of recent work on diffusion models also builds on the probability flow (PF) ODE formulation in [4]. We do not cover it in this assignment, but read Section 4.3 and Appendix D of [4] if you want to learn more!

5 Part 4: Rectified Flow — Theory

So far, we have studied diffusion models through the lens of iterative denoising (DDPM) and stochastic differential equations. In this part, we turn to **Rectified Flow** [7], a conceptually simpler generative modeling framework that learns to transport noise to data along *straight-line* trajectories. Rather than reversing a complex noising SDE, rectified flow directly regresses a velocity field that moves particles from a source distribution π_0 (typically $\mathcal{N}(0, \mathbf{I})$) to a target distribution π_1 (the data) in one shot.

Background

Given paired samples $(X_0, X_1) \sim \pi_0 \times \pi_1$, define the linear interpolation

$$X_t = (1-t)\, X_0 + t\, X_1, \quad t \in [0,1].$$

Rectified flow learns a velocity field $v_\theta(x,t)$ by minimizing

$$\mathcal{L}_{\mathrm{RF}}(\theta) = \int_0^1 \mathbb{E}\Big[\|(X_1 - X_0) - v_\theta(X_t, t)\|^2\Big]\, dt. \tag{RF}$$

At inference, one solves the ODE

$$\frac{dX_t}{dt} = v_\theta(X_t, t), \quad X_0 \sim \pi_0,$$

starting from noise and integrating forward to $t = 1$ to obtain a sample from (approximately) π_1.

Problem 4.1: Optimal Velocity Field

Show that the minimizer of $\mathcal{L}_{\mathrm{RF}}(\theta)$ over all measurable functions is the conditional expectation

$$v^*(x,t) = \mathbb{E}[X_1 - X_0 \mid X_t = x]\,.$$

Hint: Differentiate inside the expectation and set the functional derivative to zero.
Deliverable: Your proof.

Problem 4.2: Connection to Score Matching

Show that if $X_0 \sim \mathcal{N}(0, \mathbf{I})$ and p_t is the marginal density of X_t, then the optimal velocity field satisfies

$$v^*(x,t) = \frac{x - \mathbb{E}[X_0 \mid X_t = x]}{t},$$

and relate $\mathbb{E}[X_0 \mid X_t = x]$ to the score function $\nabla \log p_t(x)$ via Tweedie's formula (Problem 2.5). What does this tell you about the relationship between rectified flow and score-based diffusion models?
Deliverable: Your derivation and a 2–3 sentence discussion.

Problem 4.3: Straightness and Reflow

A key desideratum of rectified flow is that trajectories should be *straight*: ideally $v_\theta(X_t, t) \approx X_1 - X_0$ everywhere, so that a single Euler step suffices for sampling.

(1) Define the **straightness** of a flow as

$$S = 1 - \frac{\mathbb{E}\left[\int_0^1 \|v_\theta(X_t, t)\, dt - (X_1 - X_0)\|^2\right]}{\mathbb{E}[\|X_1 - X_0\|^2]}.$$

Explain in one sentence what $S = 1$ and $S = 0$ each correspond to geometrically.

(2) The **reflow** procedure improves straightness by re-pairing: after training v_θ once, generate new pairs $(\hat{X}_0, \hat{X}_1)$ by running the ODE from fresh noise $\hat{X}_0 \sim \pi_0$ to $\hat{X}_1 = \Phi^1(\hat{X}_0)$ (where Φ^1 is the flow map), and then retrain on these new pairs. Explain intuitively why re-pairing produces straighter trajectories.

(3) What is the computational cost of one round of reflow compared to the original training, and why might you limit the number of reflow rounds in practice?

Deliverable: Answers to each of the three subparts.

Problem 4.4: Rectified Flow vs. DDPM

Compare rectified flow and DDPM along the following four axes. For each, give a 1–2 sentence answer.

(1) **Training objective.** What quantity does each model regress? How do the targets differ?

(2) **Inference cost.** DDPM typically requires many denoising steps. Why can rectified flow often use fewer steps, and under what condition can it use exactly one?

(3) **Trajectory geometry.** DDPM's reverse process follows a curved SDE path. How does rectified flow's path differ, and what is the practical implication for discretization error?

(4) **Likelihood evaluation.** DDPM's ELBO gives a variational lower bound on $\log p(x)$. Does rectified flow provide a likelihood estimate? Briefly explain.

Deliverable: Four short answers, one per axis.

6 Part 5: Code Your Own Diffusion Models

In this part, you will implement a Score-Matching diffusion model, as proposed by Song et al. [4], to generate new grayscale images. We will be using the Variance Preserving (VP) SDE that you studied in Part 3 to model the diffusion.

Colab Notebook Link

As in previous assignments, you should develop locally on an IDE, then run training on Google Colab Pro+. Submitted code that does not run without error will not be considered valid for grading.

6.1 Part A: Defining the VP SDE

Make sure you read Song21 [4] and understand how the Variance Preserving (VP) SDE is defined. Pay attention to the drift and diffusion coefficients.

Problem 4.A.i: Drift and Diffusion Coefficients

In the case of VP, specify the drift coefficient $f(x,t)$ and diffusion coefficient $g(t)$.
Deliverable: Your answer.

Problem 5.A.ii: $\beta(t)$ and $c(t)$

In the case of VP, what are $\beta(t)$ and $c(t)$?
Hint: Refer to Equations (32) and (33) in Song21.
Deliverable: Your answer.

Problem 5.A.iii: Implementation

Fill in the TODOs in the `VP` class.
Part A (iii) will be graded via Gradescope autograder. Please submit your code to "Homework 4 — Code" on Gradescope.

6.2 Part B: Defining the VP Sampling

Make sure you read Appendix E of Song21 [4] and understand how to generate samples. You will implement two sampling methods: Euler-Maruyama (EM) and Predictor-Corrector (PC) with EM as the predictor.

Problem 5.B.i: Euler-Maruyama Sampler

Go over the iteration rule. Fill in the TODOs in the `Euler_Maruyama_sampler` method. Note that the analytical solution for $\sigma(t)$ depends on $c(t)$. As a result, $x(t=1) = z \cdot \sigma(t=1)$ with $z \sim \mathcal{N}(0, \mathbf{I})$.
Deliverable: Your code submission to Gradescope.

Problem 5.B.ii: Predictor-Corrector Sampler

Go over the PC algorithm (Algorithm 5 in Song21). Fill in the TODOs in the `predictor_corrector_sampler` method.
Deliverable: Your code submission to Gradescope.

6.3 Part C: Generating Samples

We will be training our model for 50 epochs, LR $= 10^{-4}$ (filled in for you in the notebook) with the VP SDE on the FashionMNIST dataset. Feel free to experiment with early stopping and patience. Remember to save your checkpoints.

Two suggestions for $[\beta_{\min}, \beta_{\max}]$: $[0.01, 5]$, $[0.01, 10]$.

Note: training a score model for 50 epochs on FashionMNIST takes ∼10 min on an A100 GPU.

Problem 5.C.i: Dataset Visualization

We are working with FashionMNIST. Plot 64 images from the dataset to get an idea of the prior distribution. What are the different classes? What are the dimensions of each image? Include your plot in the submission PDF.
Deliverable: Your plot and a brief description of the classes and image dimensions.

Problem 5.C.ii: Training Curves

Plot your losses (log scale) over epochs. Discuss any early stopping and patience strategy you implemented. Specify your $[\beta_{\min}, \beta_{\max}]$ and any hyperparameters you modified. Include your titled plot in the submission PDF.
Deliverable: Your plot and discussion.

Problem 5.C.iii: EM Samples

Generate 64 samples with your EM sampler. Include your plot in the submission PDF.
Deliverable: Your plot.

Problem 5.C.iv: PC Samples

Generate 64 samples with your PC sampler for at least 2 different numbers of corrector steps. Include your plots in the submission PDF.
Deliverable: Your plots for each setting.

Problem 5.C.v: Qualitative Discussion

Discuss any qualitative difference you observe in your samples from using different samplers, and from varying the number of corrector steps. Referring to your plot in Problem 4.C.i, comment on what your model captures well from the prior distribution and what it struggles with.
Deliverable: A 3–4 sentence discussion.

6.4 Part D: Diffusion Models for Inverse Problems

Important

Do **NOT** attempt until you are **entirely** satisfied with your work for **all** of the rest of the assignment (Parts 1 through 5).
Minimum guidance will be provided in OH or on Piazza.

You're on your own, kid. You always have been.

This part will require you to read and understand [Song22] Solving Inverse Problems In Medical Imaging With Score-Based Generative Models.

In our toy example, our measurement matrix $\mathbf{A}$ will be an explicit inpainting matrix that replaces some pixels in an image by 0. We will set the corruption rate at 75%.

Your job will be to modify the sampling process to include conditioning on the perturbed measurements at various t, so that the reverse diffusion process generates recovered images conditioned on corrupted images.

Expectations. The expected result is a plot with 3 rows: clean images, corrupted images, and reconstructed images. The plot must include the PSNR of each reconstructed image. You should use at least 5 clean images. We wrote the plotting function for you. We also expect you to explain and justify your implementation choices. Everything (plot, explanations, justifications) must appear in your submission PDF. See the Colab notebook for more info.

7 Part 6: Code Your Own Rectified Flow

In this part you will implement rectified flow alongside the DDPM you built in Part 5. The two share the same `UNet` architecture — only the forward process, loss, and sampler differ. The table below is your implementation map; each row corresponds to one problem below.

Component	**DDPM (Part 5)**	**Rectified Flow (Part 6)**
Noising process	$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$, $\epsilon \sim \mathcal{N}(0, \mathbf{I})$	$x_t = (1-t)\, x_0 + t\, \epsilon$, $\epsilon \sim \mathcal{N}(0, \mathbf{I})$
Regression target	Noise ϵ	Velocity $\epsilon - x_0$
Training loss	$\|\epsilon - \epsilon_\theta(x_t, t)\|^2$	$\|(\epsilon - x_0) - v_\theta(x_t, t)\|^2$
Network output	Predicted noise $\hat{\epsilon}$	Predicted velocity $\hat{v}$
Sampling update	Reverse SDE step (Algorithm 2 of [1])	Euler step: $x_{t+\Delta t} = x_t + v_\theta(x_t, t)\, \Delta t$
Stochasticity	Stochastic (adds noise each step)	Deterministic ODE
Typical steps	$T = 1000$	10–100; 1 after reflow

Files to update for this section

Implement the rectified flow forward process, loss, and sampler in `scripts/train_rectflow.py`. You may import and reuse the same `UNet` from Part 5 directly; no architecture changes are needed.

7.1 Part A: Forward Process and Loss

Problem 6.A: Forward Process and Training Loss

Implement the rectified flow forward process and loss (RF) in `train_rectflow.py`, mirroring the VP forward process in `VP.py`. The two implementations are shown below for direct comparison:

```
# DDPM / VP (your Part 5 VP.py, shown for reference)
t    = sample_t(batch_size)        # t ~ Uniform{1, ..., T}
eps  = torch.randn_like(x1)
x_t  = sqrt_alpha_bar[t] * x1 \
       + sqrt_one_minus_alpha_bar[t] * eps
loss = F.mse_loss(eps_theta(x_t, t), eps)

# Rectified Flow (implement this)
t    = torch.rand(batch_size)      # t ~ Uniform(0, 1)
x0   = torch.randn_like(x1)
x_t  = (1 - t) * x0 + t * x1     # TODO: broadcast t correctly
vel  = x1 - x0                    # regression target
loss = F.mse_loss(v_theta(x_t, t), vel)
```

Train on FashionMNIST for 50 epochs using the **same hyperparameters as Part 5**. Plot both training loss curves on the same axes (log scale). Do the loss scales compare directly? Why or why not?
Deliverable: Your implementation, the combined loss curve, and a 2–3 sentence discussion.

7.2 Part B: Euler Sampler and Full Comparison

Problem 6.B: Rectified Flow Euler Sampler and Full Comparison

Implement the rectified flow Euler ODE sampler and run it alongside the DDPM samplers from Part 5. Use the **same initial noise** x_0 at each grid position across all rows.

Quantitative Comparison

Fill in the KID scores (mean ± std, 1k samples via `torch-fidelity`) for each method and step count:

Steps	Flow Matching	DDIM	DDPM EM
1			—
5			—
10			—
50			—
100			—
200			—
1000			*(baseline)*

Include all 8×8 sample grids in your writeup. Then answer the following:

1. At what step count does each method first produce recognizable FashionMNIST samples?
2. What is the minimum number of steps each method needs to match 1000-step DDPM EM quality?
3. At 100 steps, how do flow matching and DDIM compare to each other and to 1000-step DDPM EM? What does this tell you about whether the bottleneck is the model or the sampler?
4. If you wanted the best quality regardless of speed, which method and step count would you choose? If you wanted fast generation with acceptable quality?

Deliverable: Completed table, sample grids, and answers to the four questions above.

7.3 Part C: Reflow

Problem 6.C: One-Step Generation after Reflow

Apply **one round of reflow** to your rectified flow model:

1. Generate 50k pairs $(\hat{X}_0, \hat{X}_1)$ by running the trained ODE (100 Euler steps) from fresh noise.
2. Retrain on these pairs for 20 epochs using loss (RF).
3. Generate 64 samples with a *single* Euler step ($\Delta t = 1$).

Add a sixth row to the table from Problem 6.B:

Method	**Steps**	**Sample grid**	**FID**	**Time (s/64)**
Rect. Flow — Reflow	1	*include*		

How does one-step reflow FID compare to one-step rectified flow without reflow, and to 1000-step DDPM?
Deliverable: Your sample grid, completed table row, and a 2–3 sentence discussion.

7.4 Part D: Unified Qualitative Comparison

Problem 6.D: Side-by-Side Qualitative Grid

Pick **8 fixed initial noise vectors** $x_0^{(1)}, \ldots, x_0^{(8)}$ and generate one sample from each method using those same seeds. Arrange all results in a single 4×8 grid:

Method	**Seed index →**
DDPM (EM, 1000 steps)	
Rect. Flow (100 steps)	
Rect. Flow (1 step)	
Reflow (1 step)	

Comment on whether the same initial noise produces semantically similar outputs across

methods, and on any systematic differences in sharpness or artifact patterns between DDPM and the flow models.
Deliverable: Your 4×8 image grid and a 3–4 sentence discussion.

8 Part 7: Diffusion Models on a Larger Scale

Colab Notebook Link

In this part, we are going to run diffusion models on a larger scale. In particular, you will move from 28×28 grayscale FashionMNIST images to 256×256 RGB images, using OpenAI's guided diffusion codebase.

For this part, we will remove almost all scaffolding:

- You need to clone the codebase, set up the environment, and read the given instructions on your own.
- There will be no instructions on where to add your code; we will directly describe what we are looking for.
- You will have total freedom about how you want to code it — just like doing research!

Tips for using this repository.

- Generated samples will be saved to `/tmp` by default. We suggest changing `OPENAI_LOGDIR` to save to a different folder.
- If you encounter a "package not found" error, move the Python file to your working directory. For example: `!mv scripts/classifier_sample.py classifier_sample.py`

You only need to include the code for plotting in the Colab notebook. Your code can load the `.npz` file generated automatically in `save_dir` and plot the figure for each part. The runtime for each subpart is 5–10 minutes with Google Colab Pro+.

Problem 7.1: Unconditional Image Generation

Clone the guided diffusion codebase and download `256x256_diffusion_uncond.pt` to `models/`. Generate 8 random images and show them in a single 256×2048 image. Include your figure, the command you used (including `SAMPLE_FLAGS` and `MODEL_FLAGS`), and brief comments on the generated images.
Deliverable: Your figure, command, and comments.

Problem 7.2: Progressive Generation

Visualize the evolution of the image generation process. Your final image should be 256×2048 (8 columns):

- Column 1: initial random noise.
- Column 8: final generated image.
- Middle columns: evenly spaced time steps between initial and final.

See Figure 14 in [4] for an example. Include your figure, command, and comments on the progressive generation process.
Deliverable: Your figure, command, and discussion.

Problem 7.3: Noise Interpolation

Generate two instances of standard Gaussian noise z_0 and z_7, picking them so that their corresponding samples are as different as possible. Then generate 8 samples with evenly spaced interpolations

$$z_i = \left(1 - \tfrac{i}{7}\right) z_0 + \tfrac{i}{7} z_7, \quad i = 0, \ldots, 7.$$

Your final image should be 256×2048. Fix the random seed so that the initial state is the only variable. See Figure 6 in [4] for an example. Include your figure, command, and comments.
Deliverable: Your figure, command, and discussion.

Problem 7.4: Conditional Image Generation

Download `256x256_classifier.pt` to `models/`. Generate 8 random images conditioned on 8 random classes and show them in a single 256×2048 image. Include your figure, command, and brief comments on the generated images.
Deliverable: Your figure, command, and comments.

Problem 7.5: Classifier Scale Sweep

The `--classifier_scale` hyperparameter controls the strength of the classifier gradient. Pick a reasonable range and plot generated samples for 8 values (monotonically increasing). Your final image should be 512×2048 (2 rows, 8 columns):

- Each column: one `classifier_scale` value.
- Two rows: two different samples with the same `classifier_scale`.
- All 8 columns share the same initial state and random seed; only `classifier_scale` varies.

Include your figure, command, and brief comments on how classifier scale affects the samples.
Deliverable: Your figure, command, and comments.

References

[1] Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. *NeurIPS*. https://arxiv.org/abs/2006.11239.

[2] Sohl-Dickstein, J. N., Weiss, E. A., Maheswaranathan, N., & Ganguli, S. (2015). Deep Unsupervised Learning using Nonequilibrium Thermodynamics. *ICML*. https://arxiv.org/abs/1503.03585.

[3] Vincent, P. (2011). A Connection Between Score Matching and Denoising Autoencoders. *Neural Computation*, 23, 1661–1674.

[4] Song, Y., Sohl-Dickstein, J. N., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. (2021). Score-Based Generative Modeling through Stochastic Differential Equations. *ICLR*. https://arxiv.org/abs/2011.13456.

[5] Anderson, B. D. (1982). Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12, 313–326.

[6] Song, Y., et al. (2022). Solving Inverse Problems in Medical Imaging with Score-Based Generative Models. *ICLR*. https://arxiv.org/abs/2111.08005.

[7] Liu, Xingchao, Chengyue Gong, and Qiang Liu. (2023). Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow." *The Eleventh International Conference on Learning Representations (ICLR)*

[8] Obukhov, Anton, and Maximilian Seitzer. "Reliable Fidelity and Diversity Metrics for Generative Models." *arXiv preprint arXiv:2002.12728*, 2020. https://github.com/toshas/torch-fidelity.